

Teoría de Algoritmos

(75.29 / 95.06) Curso Ing. Podberezski

Trabajo Práctico Nro. 2

12 de Mayo de 2025

Grupo: **BigO**

Apellido y Nombre	Padrón	e-mail
Czerwiak, Juan Joaquin	109.640	jczerwiak@fi.uba.ar
Huzan, Hugo Javier	67.910	hhuzan@fi.uba.ar
Rivero, Julián Agustín	112.454	jarivero@fi.uba.ar
Rojo, Santiago	110.700	srojo@fi.uba.ar

Parte 1: La subasta de pizza.....	4
Soluciones.....	4
Pseudocódigo y Estructuras de Datos.....	5
Backtracking.....	6
Branch & Bound.....	7
Análisis de complejidad.....	8
Backtracking.....	8
Branch & Bound.....	10
Comparación.....	11
Ejemplo paso a paso.....	11
Programa.....	13
Complejidad Implementada vs. teórica.....	14
Parte 2: El traslado de información secreta.....	15
Solución propuesta.....	15
Pseudocódigo.....	16
Análisis de optimalidad.....	18
Complejidad.....	18
Complejidad Temporal.....	19
Complejidad Espacial:.....	20
Vértices.....	21
Aristas.....	21
Programa.....	22
Complejidad del algoritmo vs. la del programa.....	22
¿Es posible expresar su solución como una reducción polinomial?.....	23
Parte 3: Las 2 jornadas de capacitación.....	24
Interpretación del problema.....	24
El problema es NP.....	24
El problema es NP-Hard.....	24
El problema es NP-C.....	25
Set splitting problem pertenece a NP-C.....	25
Set-Splitting Problem es NP.....	26
Set-Splitting Problem es NP-Hard.....	26
Reducción de 3-SAT a NAE-3-SAT.....	26
Reducción de NAE-3-SAT a SPP.....	27
Qué pasa si se encuentra un método eficiente.....	28
Qué se puede decir de un problema que se reduce a este problema.....	28
Referencias.....	29
1º Reentrega.....	30
Parte 1: La subasta de pizza.....	30
Parte 2: El traslado de información secreta.....	33
Parte 3: Las dos jornadas de capacitación.....	34

Parte 1: La subasta de pizza

El prestigioso restaurante “Altezzoso” de 3 estrellas “Michi Inn” se especializa en la elaboración de pizzas gigantes. Una vez por año realiza una recaudación por causas caritativas. Invita a “n” de las personas mas ricas del planeta y realiza una subasta de una pizza tamaño XXXL cortada en “m” porciones ($n > m$). Los invitados tienen un presupuesto de “p” liras que pueden dividir en “ofertas” para las diferentes porciones. La pizza ocupa una mesa circular donde se encuentra una silla frente a cada porción. Quien obtiene una determinada porción debe sentarse en su lugar para comerla. Un invitado solo puede acceder a una única porción. No necesariamente quien ofrece más plata por una porción la obtiene. La decisión es única y exclusivamente potestad del restaurante. Ellos quieren maximizar la ganancia. Pero deben tener en cuenta una cuestión adicional: algunos de los invitados están enemistados con otros y no quieren sentarse a su lado. Por lo que informan que prefieren no comer a tener que soportarlos.

Nos contratan para que, en base a la información provista, determinemos qué porción asignar a cada invitado de forma tal de maximizar la ganancia cumpliendo las restricciones del problema.

Nos sugieren la siguiente la solución: Mediante backtracking determinar todas las posibles ubicaciones factibles (sin considerar las rotaciones en la mesa). Luego, por cada una de ellas calcular las ganancias de cada una de las rotaciones posibles.

Queremos proponer una solución alternativa utilizando branch and bound.

Soluciones.

Backtracking

La solución mediante **backtracking** se basa en la sugerencia del restaurante. Partiendo de la lista de ofertas por porción de cada invitado, y la de incompatibilidades entre invitados, se construyen recursivamente todas las asignaciones posibles de invitados a porciones, sabiendo que pueden quedar asientos vacíos para maximizar la ganancia.

Para evitar que una nueva asignación corresponda a la “rotación” de un estado previamente contemplado, el primer invitado es **fijado en su posición**, asignando en las otras posiciones solo invitados cuyo índice sea mayor al del primero.

Para respetar las restricciones de enemistad impuestas, se implementa una **función límite** para analizar la **propiedad de corte**. Esta función verifica que la persona que se sentó anteriormente no está enemistada con la persona que está siendo evaluada, y viceversa. También verifica que la primera y última persona en ser ubicada no estén enemistados entre sí.

Una vez obtenidos todos los estados posibles, se irá rotando a cada uno de ellos para quedarse con la ubicación que dé la mayor ganancia posible entre todos los estados.

Branch & Bound

Para la solución mediante **Branch & Bound**, se parte del algoritmo anterior, conservando la misma **función límite** para analizar la propiedad de corte en cada paso.

Para recorrer el árbol con la estrategia **best-first search (BeFS)** se utiliza un **heap de máximos**, lo que permite extraer en tiempo $O(\log k)$, siendo k el tamaño del heap¹, la solución parcial más prometedora.

La cota superior de ganancia se estima mediante una **función costo**, que posee una lista con la mejor oferta posible por cada porción de pizza. Esto se debe a que el mejor caso posible es cuando se asigna cada porción a su mejor postor, sin enemistades que invaliden esta selección.

En cada solución parcial se calcula el valor estimado restante teniendo en cuenta las sillas que aún no han sido ocupadas, y asumiendo que puede corresponder al mejor caso posible.

Si la suma de la ganancia real acumulada, y el valor estimado restante, es menor al valor máximo encontrado hasta el momento, entonces la rama se poda y no se sigue evaluando. En cambio, si se logra sentar a todos los invitados, entonces se actualiza este valor máximo parcial.

Cabe destacar que esta solución contempla **todas las configuraciones posibles directamente en el árbol**, incluyendo las **rotaciones**, lo cual evita tener que rotar cada estado posteriormente.

Pseudocódigo y Estructuras de Datos

Función límite:

```
supera_restricciones(Seleccionados, Candidato, m, Restricciones):  
    Si el último de Seleccionados está entre las Restricciones del Candidato:  
        Retornar Falso  
    Si el Candidato está entre las Restricciones del último de Seleccionados:  
        Retornar Falso  
    Si el Candidato ocuparía el último lugar de los seleccionados: # |seleccionados| == m-1  
        Si el primero de los Seleccionados está entre las Restricciones del Candidato:  
            Retornar Falso  
        Si el Candidato está entre las Restricciones del primero de Seleccionados :  
            Retornar Falso  
    Retornar Verdadero
```

¹ [Stirling Approximation. https://en.wikipedia.org/wiki...](https://en.wikipedia.org/wiki/Stirling_approximation)

Backtracking

```
backtracking(m, n, restricciones):  
    sea resultados una lista vacía  
    para i de 0 a n-1:  
        llamar bt_rec([i], m, n, restricciones, resultados)  
    retornar resultados
```

```
bt_rec(seleccionados, m, n, restricciones, resultados):  
  
    si longitud(seleccionados) es igual a m:  
        agregar seleccionados a resultados y retornar  
  
    llamar a bt_rec agregando un posible asiento vacío  
  
    para i desde seleccionados[0] + 1 hasta n:  
        si i ya se encuentra en seleccionados: #El evaluado ya se sentó  
            podar  
        si i no supera las restricciones:  
            podar  
    llamar bt_rec con el siguiente de seleccionados
```

Función de máxima ganancia con rotaciones:

```
buscar_max_en_rotaciones(factibles, ofertas):  
    sea máximo inicialmente un valor negativo  
    sea seleccionados una lista vacía  
  
    para cada estado factible en factibles:  
        repetir |factible| veces:  
            ganancia_actual es 0  
            para cada porción desde la primera hasta |factible|:  
                sea invitado el invitado sentado en la porción  
                si el invitado no es una silla vacía:  
                    ganancia_actual += oferta hecha del invitado por la porción  
            si ganancia_actual > máximo:  
                máximo = ganancia_actual  
                seleccionados = copia de factible  
            rotar factible a la izquierda (mover primer elemento al final)  
    retornar máximo y seleccionados
```

Estructuras utilizadas

Se utilizan **Listas** para almacenar a los invitados como índices asignados a partir de los archivos provistos, las ofertas de cada uno de ellos, las restricciones de enemistad, y las soluciones parciales y finales.

Branch & Bound

Función de branch & bound:

```
branchbound(m, n, restricciones, v_e_inicial, f_estimado):  
    crear un heap de máximos que guarda tuplas (v_estimado, v_real, lista de personas)  
    sea valor máximo actualmente 0  
    sea la selección máxima una lista vacía  
  
    por cada persona:  
        calcular valor estimado v_e_p de sentarlo en la primera silla con f_estimado  
        calcular el valor real v_r_p de sentarlo en esa silla  
        si el valor estimado es menor al máximo hasta el momento, podar  
        si el valor real es mejor al máximo actual:  
            actualizar valor máximo y selección máxima  
            insertar la tupla (v_e_p, v_r_p, [persona]) al heap  
  
    mientras el heap no esté vacío  
        extraer del heap  
        si las sillas están llenas o el valor estimado es menor al máximo, podar  
        incluir al heap posible caso de silla vacía  
        para cada invitado i que no haya sido asignado todavía:  
            si no cumple las restricciones, podar  
            calcular valor estimado de sentar a i en la silla actual con f_estimado  
            si el valor estimado de sentar a i es menor que el máximo, podar  
            calcular el valor real de sentar a i  
            sumar a i a la lista de personas sentadas  
            si el valor real con i supera al máximo:  
                actualizar valor máximo y selección máxima  
                reinsertar al heap teniendo en cuenta que se sentó i  
  
    retornar valor y selección máximos
```

Función de valor estimado:

```
f_estimado(v_r_anterior, persona, silla, ofertas, sel, m, n):  
    Crear un conjunto con los invitados ya asignados  
    Inicializar v_e como el valor acumulado v_r_anterior  
    Crear una lista de maximas ofertas de tamaño sillas restantes  
  
    calcular valor máximo posible teniendo en cuenta las sillas e invitados sin asignar  
  
    Sumar al valor v_e la suma de todos los valores en máximas ofertas  
  
    Si la silla está vacía, retornar v_e  
    Si no, retornar v_e más la oferta de persona para la silla actual
```

Estructuras utilizadas

Se utilizan **Listas** para los mismos roles que el algoritmo de backtracking.

También se usa un **heap de máximos** para gestionar las soluciones parciales del algoritmo de Branch & Bound, permitiendo extraer la solución parcial más prometedora. cada nodo del heap contiene una **tupla** de la forma (v_e_sel, v_r_sel, sel) donde:

- **v_e_sel** = cota superior de ganancia estimada para la solución parcial
- **v_r_sel** = ganancia real acumulada hasta el momento
- **sel** = asignación de sillas hasta el momento (admitiendo sillas vacías)

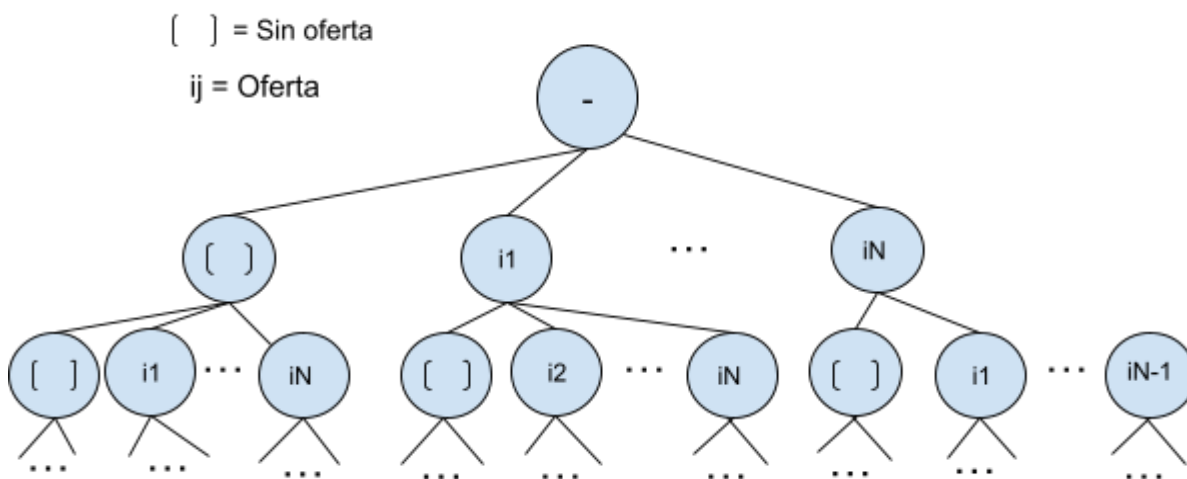
Análisis de complejidad

Para ver la complejidad del algoritmo se puede observar el árbol de estados del algoritmo, este en este árbol cada nivel es una porción, la primera porción puede llevarla o nadie o uno de los “n” invitados. Así sucesivamente, en cada nivel se quita al que ya se llevó su porción.

En la primera elección tenemos las N ofertas más la opción de no tomar ninguna oferta.

En la segunda elección tenemos todas las opciones menos la que elegimos.

Esto nos muestra un comportamiento factorial, en principio la complejidad que nos trae el árbol de estados del problema es $O(n!)$, esto es todas las formas de ordenar los “n” invitados, considerando que se recorre todo el árbol



Debe tenerse en cuenta que el número de invitados “n” es mayor a la cantidad de porciones “m”, por eso podemos calcular la cantidad de formas de ordenar a los invitados entre las porciones como la cantidad de combinaciones de “n” tomados de “m”. esto es $\frac{n!}{(n-m)!}$, además a n se suma 1 que es la posibilidad de no vender la porción es decir $\frac{(n+1)!}{(n+1-m)!}$. Se puede observar que mientras más similares son “n” y “m”, mas se acerca a $(n+1)!$. Por eso se propone para acotar superiormente la complejidad con $(n+1)!$, sabemos que no superará este valor dado que el mayor valor posible de “m” es $m = n - 1$, para este caso tendríamos, $\frac{n+1!}{(n+1-(n-1))!} \rightarrow \frac{n+1!}{2!} \rightarrow \frac{(n+1)(n!)}{2!}$ es decir $O(n!n)$. Podemos acotar la complejidad superiormente con $O(n!n)$.

Backtracking

Complejidad temporal:

Para la solución por Backtracking se utiliza una función límite que en cada uno de los estados del problema o nodos, poda los que no complen con ciertas restricciones, estas restricciones son:

- Verificación de no haber sido seleccionado:
Los seleccionados se guardan en un diccionario, se verifica en $O(1)$.
- Verificación de no ser odiado por el anterior:
Esto es $O(1)$ ya que las restricciones están almacenadas en diccionarios.
- Verificación de no odiar al anterior:
De la misma forma por estar implementado con diccionarios verifica no odiar al anterior en $O(1)$.

Así la poda que realiza backtracking es $O(1) + O(1) + O(1) = O(1)$

Podemos concluir que realizar la función backtracking es $O(n! n)$

Luego de esto se rotan todos los ordenamientos válidos:

Por cada resultado posible se evalúa las “m” posibles rotaciones, esto tiene un costo de $O(m^2)$

Es decir $O(n! n m^2)$

Entonces al utilizar backtracking la complejidad es realizar la exploración de las posibles combinaciones mas probar cada rotación para cada combinación.

Esto es $O(n! n) + O(n! n m^2) = O(n! n m^2)$

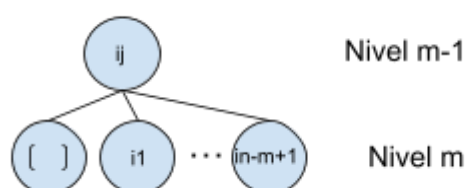
Complejidad espacial:

Se almacenan las ofertas $O(nm)$ y restricciones $O(n^2)$.

Además se almacenan todos los resultados posibles, esto es todos los nodos hoja del árbol , para esto vemos cuántos son los nodos hoja del árbol.

[] = Sin oferta

ij = Oferta



Los nodos hoja del árbol de estado son mayoría, ya que, contemplando que $n > m$ y que se puede dejar el lugar vacío, al llegar a la última elección, esto es la elección “m”, tenemos la opción vacío más los $n - m + 1$ invitados que no están seleccionados en este estado solución.

Observando esto podemos evaluar cuántos nodos del árbol están en el último nivel “m”.

Supongamos que en el último nivel tenemos un número “k” de nodos hoja, en su nivel inmediatamente superior tendríamos $k / (n - m + 2)$ con $n > m$, así sucesivamente hasta el nivel 0, pero a cada nivel que subimos en el nivel superior hay una opción más al elegir. En el sentido ascendente del árbol se achica cada vez más rápido. Teniendo en cuenta que un árbol binario completo tiene la mitad de nodos hoja, este árbol tiene al menos la mitad de nodos hoja si no más. Al evaluar la complejidad espacial consideramos que se almacenan en el orden de $n! n$ resultados.

En los $n! n$ nodos se almacena un resultado de largo “m”, esto es $O(n! n m)$.

También se almacenan los seleccionados $O(m)$.

Esto es $O(nm) + O(n^2) + O(n!nm) + O(m) = O(n!nm)$

Branch & Bound

Complejidad temporal:

Para la resolución por Branch & Bound se utiliza una función obtener valor estimativo y una función obtener valor real.

Se guarda la mejor oferta para cada porción, las cuales se usarán en la función de corte como cota superior. Esto es por cada una de las “m” porciones se recorren las “n” ofertas y se elige la mayor, es decir $O(nm)$.

Se llama a Branch & Bound que utiliza un heap para almacenar los resultados parciales ordenados por su cota superior, obteniendo así los de mayor cota primero.

Dentro de cada estado de solución se calcula la función límite y costo para cada uno de los hijos.

Esto es, para “n” hijos en el peor de los casos:

- Verificar restricciones: $O(1)$, igual que en backtracking
- Calcular función costo o estimación de ganancia tiene un costo de $O(nm)$, ya que calcula las máximas “m” ofertas, para los “n” no seleccionados en el peor de los casos.
- Calcular la ganancia actual constante porque se almacena el valor real y se suma la elección actual, es $O(1)$
- Insertar en el heap es $O(\log(k))$, siendo k la cantidad de elementos en el heap. En el heap como máximo hay todas las combinaciones posibles, es decir $O(n!n)$; por lo que insertar es $O(\log(n!n))$, que por aproximación de Stirling², es igual a $O(n \log(n))$

Es decir que tenemos $n * (O(1) + O(nm) + O(1) + O(n \log n)) = O(n^2m)$

Considerando que esto se realiza por cada estado del problema: $O(n!n^3m)$

Complejidad espacial:

Se almacenan las ofertas $O(nm)$ y restricciones $O(n^2)$.

Además se almacena la mejor oferta para cada porción, con “m” porciones, esto es $O(m)$.

Dentro del Branch & Bound se almacena todo el Heap, el cual almacena en el peor de los casos todos los estados del problema $O(n!n)$. Dentro de cada uno de ellos guarda dos enteros $O(1)$ y la solución hasta ese momento que en el peor de los casos es $O(m)$.

Esto es una complejidad espacial $O(n!nm)$

² [Stirling Aproximation. https://en.wikipedia.org/wiki...](https://en.wikipedia.org/wiki/Stirling%20approximation)

Comparación

	Temporal	Espacial
BackTracking	$O(n! \cdot n \cdot m^2)$	$O(n! \cdot n)$
Branch & Bound	$O(n! \cdot n^3 \cdot m)$	$O(n! \cdot n \cdot m)$

Por más que parezca que *Branch & Bound* tenga mayor complejidad temporal, esto a fines prácticos no es así, ya que este algoritmo busca podar cuanto antes para así visitar posteriormente menos nodos. Además, si se utiliza un algoritmo que visita por *best first*, se encuentra el valor estimativo más alto hasta el momento, permitiendo obtener un valor real máximo más rápido para podar aún más y guardarse en el heap menos nodos.

Un ejemplo práctico es utilizar una función para obtener el estimativo en complejidad temporal constante o usar una función con mayor costo computacional pero mejor poda. Resultó que podar más daba un mayor rendimiento.

Ejemplo paso a paso

Se muestra un ejemplo simple con 3 porciones, y 3 invitados.

Ofertas:

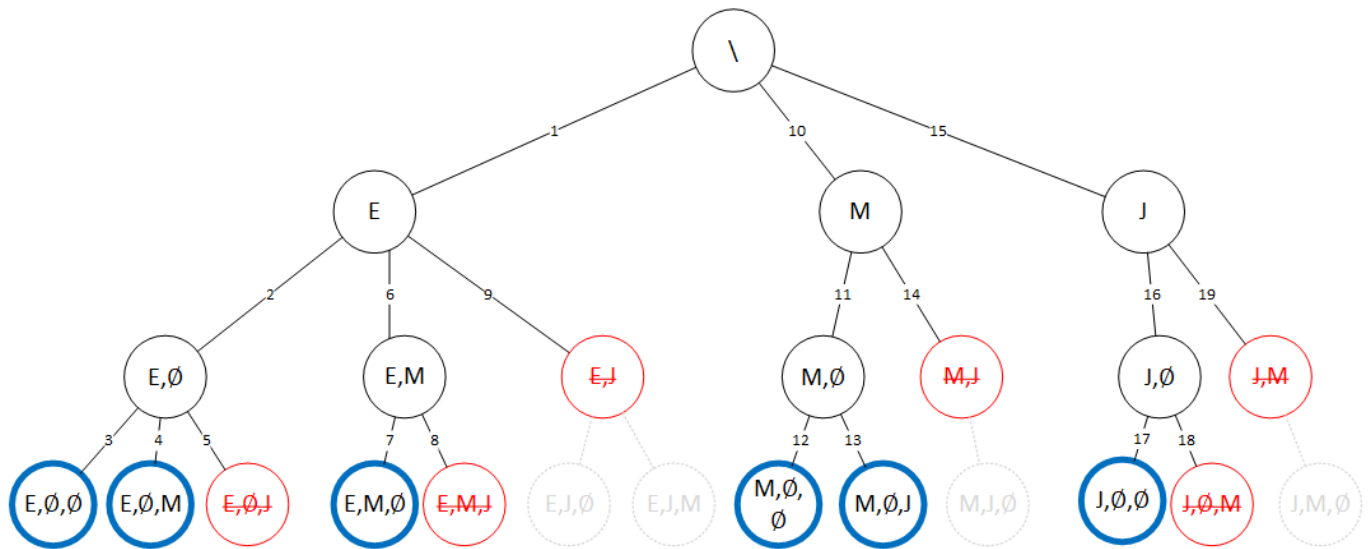
	Porción 1	Porción 2	Porción 3
Elon	10	5	10
Mark	0	0	25
Jeff	7	7	8

Restricciones:

Elon	Jeff
Mark	
Jeff	Mark

Backtracking

Exploración del Árbol de Estados



○Nodos Podados por enemistades (función de límite)

Nodos Seleccionados

Los números en las líneas indican el orden en que se analizan los estados

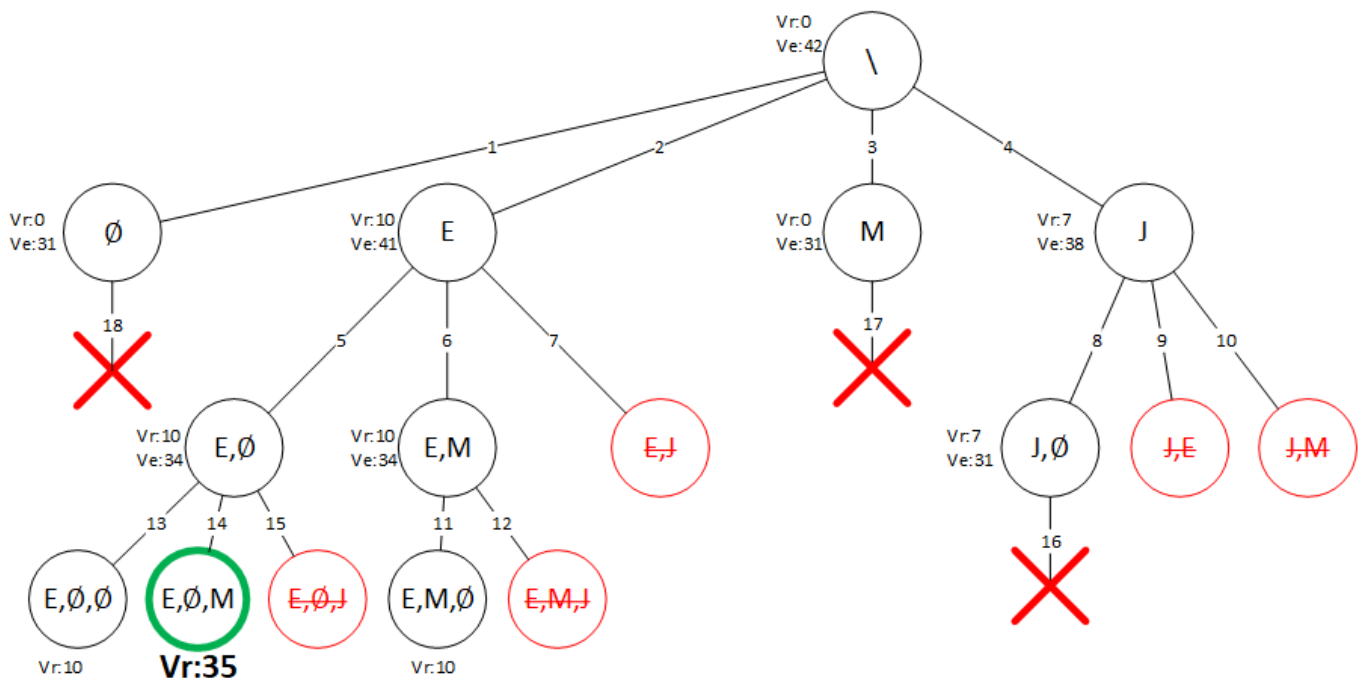
Análisis de las rotaciones entre los estados seleccionados:

Seleccionados	Mejor Rotación	Ganancia
E, $\emptyset$, $\emptyset$	E, $\emptyset$, $\emptyset$	$10 + 0 + 0 = 10$
E, $\emptyset$, M	E, $\emptyset$, M	$10 + 0 + 25 = 35$
E, M, $\emptyset$	$\emptyset$, E, M	$0 + 5 + 25 = 30$
M, $\emptyset$, $\emptyset$	$\emptyset$, $\emptyset$, M	$0 + 0 + 25 = 25$
M, $\emptyset$, J	$\emptyset$, J, M	$0 + 7 + 25 = 32$
J, $\emptyset$, $\emptyset$	$\emptyset$, $\emptyset$, J	$0 + 0 + 8 = 8$

Finalmente la mejor asignación corresponde a E, $\emptyset$, M (**Elon, Vacío, Mark**) con una ganancia de **35 Liras**.

Branch & Bound

Árbol de Estados:



○ Nodos podados por Enemistades (función límite)

✗ Poda por función costo

Los números en las líneas indican el orden en que se analizan los estados

Vr: Valor Real

Ve: Valor Estimado

Programa

Requisitos:

- Python 3

Ejecución Backtracking:

```
python3 subasta_bt.py archivoOfertas archivoRestricciones
```

Ejecución Branch & Bound:

```
python3 subasta_bb.py archivoOfertas archivoRestricciones
```

Complejidad Implementada vs. teórica.

Para la implementación del algoritmo de *branch and bound* por best first se utilizó un heap personalizado que alterna entre un heap de máximos y mínimos según se especifique. Este heap personalizado es a fin de cuentas un atajo a `heapq`, una implementación de una cola de prioridad que viene con python.

Se puede ver la implementación de `heapq`³ que es un heap con complejidades $O(\log(n))$ para insertar o extraer de la cola de prioridad. Dado esto, la complejidad teórica calculada no cambia.

³ [Python heapq — Heap queue algorithm https://docs.python....](https://docs.python.org/3/library/heapq.html)

Parte 2: El traslado de información secreta

Contamos con un “ n ” agentes secretos que deben realizar el traslado de información importante. Cada agente se encuentra distribuido en diferentes ciudades (algunos podrían estar en la misma). Cada uno de ellos debe llevar la información a alguno de los “ n ” centros de investigación. Estos centros se encuentran cada uno en diferentes ciudades. Para viajar utilizarán como medio de transporte la red de líneas aéreas disponibles. Existe un subconjunto de ciudades seguras en las que podrían realizar escalas. Como medio de seguridad quieren que no más de “ s ” agentes pasen por la misma ciudad. Todas los vuelos son bidireccionales (es decir si existe el vuelo de la ciudad A a la B. También existe el vuelo de la ciudad B a la A)

Determinar si es posible que los agentes puedan realizar el envío y en caso positivo determinar qué ruta debe seguir cada uno de ellos.

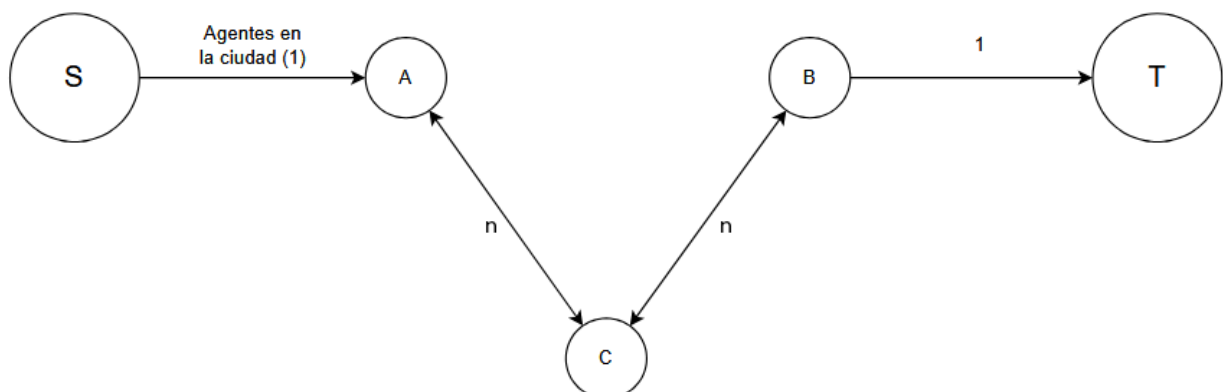
Solución propuesta.

Para determinar si es posible que los n agentes realicen el envío podemos usar redes de flujo. Como ejemplo para la explicación de la solución vamos a usar un caso sencillo: una ciudad A, que contiene un agente y 0 centros de investigación, una ciudad B que contiene 0 agentes y 1 centro de investigación, y una ciudad C que no tiene agentes ni centros, y que sirve como escala entre A y B.

Como primera instancia, podemos modelar un grafo, dirigido y pesado, de la siguiente manera: Como estamos resolviendo este problema mediante redes de flujo, debemos agregar una fuente S y un sumidero T. La fuente estará conectada a las ciudades que contengan agentes (en nuestro ejemplo, A). El peso de la arista que conecta a la fuente con estas ciudades, será la cantidad de agentes que empiezan en dicha ciudad. Por otro lado, al sumidero estarán conectadas todas las ciudades que contengan un centro de investigación. El peso de esa arista siempre será 1 ya que no puede haber más de un centro por ciudad. Las ciudades que no contienen agentes ni poseen centros de investigación estarán, visualmente, en el medio.

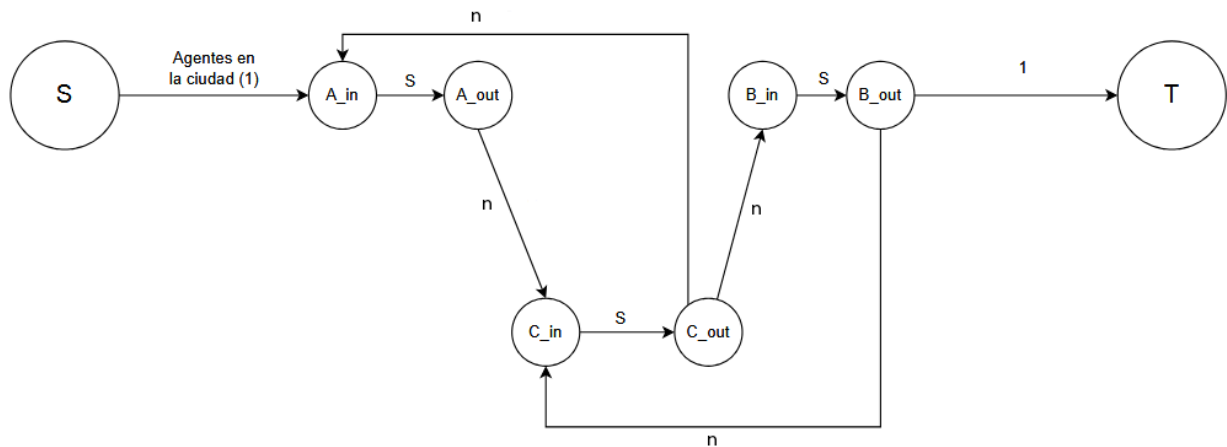
Entre ciudades que contengan una ruta aérea que las conecta, sus aristas serán bidireccionales, y su peso será n , ya que el flujo nunca superará el número de agentes disponibles.

En la siguiente imagen se puede ver una visualización del grafo de esta primera instancia.



Nos falta representar el límite máximo de agentes por ciudad “S”. Para eso, debemos separar cada nodo en dos, creando un nodo de entrada y otro de salida. Cada ciudad tendrá un par de estos nodos, que siempre están conectados por una arista que va del nodo de entrada al de salida, con un peso “S”, limitando así la cantidad de agentes que pueden haber en una ciudad.

La conexión entre dos ciudades, que en el grafo anterior eran representadas como una arista bidireccional, ahora es separada en dos aristas direccionales, donde el nodo de salida de una ciudad siempre irá hacia el nodo de entrada de la otra, conservando el peso n. Por otro lado, las aristas que salen de la fuente siempre deben ir a un nodo de entrada, y las aristas que van hacia el sumidero siempre deben provenir de un nodo de salida.



De esta manera, podemos aplicar el algoritmo de Ford-Fulkerson a este grafo para encontrar el flujo máximo. Si este flujo es igual a “n”, significa que todos los centros fueron visitados por un agente, y es posible realizar el envío. Una vez determinado eso, reconstruimos el camino utilizado de la fuente al sumidero para determinar la ruta posible.

Pseudocódigo.

Creación del Grafo

```
Partiendo de un Grafo vacío
Dado S máximo de espías por ciudad y N número de espías y centros

Por cada Vuelo disponible
  Por cada Ciudad del Vuelo
    Si la Ciudad no existe
      Crear vértices ciudad_in, ciudad_out
      Crear un eje de peso S desde ciudad_in hacia ciudad_out
    Crear nuevo eje de peso N desde ciudad1_out hacia ciudad2_in
    Crear nuevo eje de peso N desde ciudad2_out hacia ciudad1_in

Crear un nuevo vértice Fuente
```

```
Por cada Agente
  Si existe un eje desde la Fuente hacia ciudad_in en que inicia el agente
    Aumentar en 1 el peso del eje
  Si no
    Crear nuevo eje de peso 1 desde Fuente hasta ciudad_in en que inicia el agente

Crear un nuevo vértice Sumidero
Por cada Centro
  Crear nuevo eje de peso 1 desde ciudad_out correspondiente hasta Sumidero
```

Edmons-Karp

```
edmondsKarp(Grafo, fuente, sumidero):
  crear un grafo_residual a partir de Grafo (copiando capacidades)

  inicializar flujo_máximo en 0

  mientras exista camino de aumento desde fuente a sumidero en grafo_residual(usando BFS):

    sea w el cuello de botella del camino de aumento elegido

    para cada arista[u-v] en el camino de aumento elegido:
      reducir su peso en w
      si en grafo_residual existe una arista [v-u]:
        incrementar su peso en w // flujo inverso
      si no:
        agregarla con peso w

    incrementar flujo_maximo en w

  retornar flujo_maximo
```

Reconstrucción

```
reconstrucción(grafo_original, grafo_residual, espías, centros):

  sea lista_flujo una lista de tuplas inicialmente vacía
  para cada arista[u-v] en el grafo_original
    sea su flujo la diferencia entre la capacidad del grafo original y el residual
    si u y v corresponden a distintas ciudades y su flujo no es nulo:
      agregar la tupla (u,v,flujo) a la lista_flujo

  Sea rutas una lista inicialmente vacía

  para cada espia en la lista espías:

    sea ruta_actual una lista vacía
    destino = espia
```



```
mientras destino no sea el Sumidero:
    origen = destino
    Sea (u, v, flujo) la primera tupla con mismo origen de lista_flujo

    agregar origen_actual a ruta_actual
    decrementar, en lista_flujo, en 1 el flujo de la tupla obtenida

    destino = v

    agregar el centro correspondiente a origen a la ruta

    agregar ruta_actual a la rutas
retornar rutas
```

Análisis de optimalidad.

La solución propuesta modela el problema como una red de flujo con capacidades enteras, donde cada agente representa una unidad de flujo partiendo desde un nodo fuente hasta un nodo sumidero pasando en el último paso por una ciudad correspondiente a un centro de investigación. Las restricciones del problema (capacidad máxima s por ciudad, escalas solo en ciudades seguras, conectividad a través de vuelos) se representan mediante transformaciones usuales en problemas de flujo, como la duplicación de nodos para limitar la capacidad de paso por cada ciudad.

Dado que el modelo es una representación fiel del problema original y que se resuelve utilizando el algoritmo de Edmonds-Karp cuya corrección y optimalidad fue mostrada en las clases teóricas de la cátedra⁴:

- Si existe una forma de enrutar a todos los agentes cumpliendo las restricciones, el algoritmo encontrará dicha solución.
- Si no existe, el flujo máximo será menor que n , indicando correctamente que el problema no tiene solución factible.

Por lo tanto, la solución propuesta es óptima dado que en todos los casos devuelve la respuesta correcta.

Complejidad

De la entrada y el enunciado podemos obtener estas variables para la formulación de la complejidad temporal y espacial:

- n = cantidad agentes = cantidad centros.
- s = restricción por ciudad.
- c = cantidad de ciudades.

⁴ [Podberezski, V.. "Apunte TDA -. Redes de Flujo...](#)

Complejidad Temporal

Para la complejidad se tomará en cuenta la implementación de un grafo de diccionarios de diccionarios.

Creación del grafo

Para la creación del grafo se parte de un grafo vacío cuya creación es en tiempo constante.

Se va procesando una entrada de conexión entre dos ciudades por línea:

1. Si no existe la primera ciudad, se agregan dos nodos con una conexión entre estos en tiempo constante.
2. Sucede de la misma manera con la segunda ciudad.
3. Se agrega una conexión del nodo de salida de la primera ciudad al nodo entrada de la otra en tiempo constante

A lo sumo, cada ciudad se conecta con la otra, siendo una entrada de longitud $c \cdot (c - 1) = c^2 - c$.

Luego se crea una fuente en tiempo constante.

Se recorre a todos los n espías y por cada uno de estos se suma una conexión entre la fuente y la entrada de la ciudad en la que se encuentra el espía en tiempo constante.

Luego se crea el sumidero en tiempo constante.

Por todos los n centros se crea una conexión entre la salida de la ciudad en el que se encuentra y el sumidero.

Así, la complejidad temporal está dada por $T(n) = O((c^2 - c) + n + n) = O(c^2 - c + 2n)$.

Como n son la cantidad de espías y centros, cada centro está en una ciudad diferente, se sabe que $n \leq c$ y podemos acotar superiormente la complejidad por $T(n) = O(c^2 - c + 2c) = O(c^2 + c)$ y quedarnos con el término dominante:

$$T(n) = O(c^2)$$

Edmons-Karp

La complejidad temporal del algoritmo de está dada como: $O(V \cdot E^2)$.

Donde **V**: cant. vértices, **E**: cant. de ejes.

En nuestro caso:

$$V = 2c + 2 \text{ ([ver complejidad espacial del grafo](#))}$$

$$E = c^2 + 2n \text{ ([ver complejidad espacial del grafo](#))}$$

$$\text{Por lo cual: } O(V \cdot E^2) = O((2c + 2)(c^2 + 2n)^2) = O(2c^5 + 8c^3n + 8cn^2 + 2c^4 + 8c^2n + 8n^2)$$

Como se demostró anteriormente, $n \leq c$. Dado esto se puede acotar superiormente: $O(V \cdot E^2) = O(2c^5 + 8c^4 + 8c^3 + 2c^4 + 8c^3 + 8c^2) = O(2c^5 + 10c^4 + 16c^3 + 8c^2)$, a lo que nos quedamos con el término más significativo:

$$T(c) = O(c^5)$$

Reconstrucción

Armado de la lista de flujo:

- Se recorren todas las aristas del grafo original: $O(c^2)$
- Se hacen operaciones de comparación $O(1)$ e inserción en la lista $O(1)$

Para la reconstrucción de las rutas, se recorre desde los nodos correspondientes a las ciudades donde inician los agentes hasta el sumidero. Como máximo hay n agentes, por lo tanto se realizan n recorridos.

Cada recorrido visita, en el peor caso, todas las ciudades del grafo.

En cada paso del recorrido:

- Puede ser necesario inspeccionar todos los adyacentes del nodo actual para encontrar uno con flujo positivo. Como una ciudad puede estar conectada con hasta $O(c - 1 + \text{sumidero})$ otras ciudades, el grado máximo de un nodo es $O(c)$.
Dado que comprobar si hay flujo es constante, inspeccionar a todos los adyacentes del nodo actual es $O(c)$.
- Se realizan operaciones constantes de listas y diccionarios.

El costo de cada recorrido es $O(c^2)$. Se realiza n recorridos. Entonces, la complejidad de obtener las rutas es $O(n \cdot c^2)$

Dado que la obtención de rutas tiene mayor complejidad que la copia del flujo, la complejidad final de la reconstrucción es: $O(n \cdot c^2)$.

Sumarizando:

- Creación del Grafo: $O(c^2)$
- Edmons-Karp: $O(c^5)$
- Reconstrucción: $O(n \cdot c^2)$

Finalmente prevalece la complejidad del algoritmo Edmons-Karp. Por lo cual la complejidad espacial de todo el proceso queda: $O(c^5)$

Complejidad Espacial:

A lo largo de todo el programa se utiliza un grafo con el que transformamos el problema en un problema de flujo, un grafo residual para el algoritmo de resolución y una lista de flujos para la reconstrucción. A continuación se detalla el espacio que consume cada una:

El grafo formado en una instancia está compuesto por:

Vértices

- Por cada ciudad se agrega dos nodos ciudades, uno para la entrada y otro para la salida a otras ciudades.
- Se crea una fuente y un sumidero.

Así, la cantidad de vértices en el grafo dado por los parámetros de la entrada es $v = 2c + 2$.

Aristas

- Por cada entrada y salida de una ciudad se crea una arista.
- Cada ciudad puede estar conectada con todas las demás ciudades.
- Cada espía puede estar en ciudades diferentes, luego la fuente se conecta con estas n ciudades.
- Cada centro está en ciudades diferentes, luego estas n ciudades se conectan al sumidero.

Así, la cantidad de aristas en el grafo dado por los parámetros de la entrada es $e = c + c * (c - 1) + n + n = c^2 + 2n$.

Si tomamos en cuenta una implementación de una lista de adyacencias:

$S(v, e) = O(v + e)$, podemos reemplazar v y e por lo dicho quedando $S(c, n) = O(2c + 2 + c^2 + 2n)$.

Como se detalló con anterioridad, $n \leq c$ y podemos acotar superiormente la complejidad por $S(c) = O(2c + 2 + c^2 + 2c) = O(c^2 + 4c + 2)$ y quedarnos con el término dominante:

$$S(c) = O(c^2)$$

El grafo residual es una copia del original al que se le duplican las aristas: $S(c) = O(2c^2) = O(c^2)$.

La lista de flujos es una lista que contiene el flujo por cada arista en el grafo. Con anterioridad definimos la cantidad de aristas que puede haber. Así, su complejidad espacial es $S(c) = O(c^2)$

Luego la complejidad espacial total queda dada por la suma de estos tres: $S(c) = O(3c^2)$

$$S(c) = O(c^2)$$

Programa

Requisitos:

- Python 3

Ejecución:

```
python3 traslado.py n s archivoCiudades archivoEspias
```

Complejidad del algoritmo vs. la del programa.

En la implementación en Python:

Armado del Grafo.

- Se recorre la lista de vuelos, la cual puede tener como máximo $c^2 - c$ vuelos
 - Por cada vuelo se insertan como máximo 2 nodos ciudades y 3 ejes.
- Se recorre la lista de centros y espías (longitud $2c$)
 - Por cada uno se inserta como máximo un eje.

La inserción de de nodos o ejes es $O(1)$ dado que el grafo se implementó como un diccionario de adyacencias.

Por lo tanto el la implementación del armado tiene, al igual que la teórica, complejidad temporal $O(c^2)$.

Algoritmo Edmons-Karp

El algoritmo itera hasta no encontrar camino en el flujo residual por BFS.

El número total de iteraciones puede acotarse dado que:

- Cada arista puede saturarse y causar aumento en distancia de BFS a lo sumo $O(V)$ veces⁵.

Por lo tanto la cantidad total de iteraciones es $O(V.E)$ y esto con nuestras variable resulta: $O(c^3)$

Cada iteración ejecuta una **búsqueda BFS** para encontrar un camino (es el de aumento):

- Cada BFS explora a lo sumo $E \simeq c^2$ aristas.
- Las operaciones *pop()* y *append()* en **deque** de **Python** tiene comp. temporal $O(1)$

Finalmente, multiplicando el costo de cada BFS por el número total de iteraciones, la complejidad temporal del algoritmo queda: $O(c^3) \cdot O(c^2) = O(c^5)$

⁵ [Introduction to Algorithms – Cormen, Leiserson, Rivest, Stein “...](#)

Reconstrucción

Armado de la lista flujo:

- Se recorren todos los nodos (cantidad = $2c + 2$)
 - Por cada nodo se recorren los adyacentes (max $c+1$ adyacente)
 - Se realizan comparaciones y agregados en la lista (comp. temp. $O(1)$)

Por lo tanto el armado de la lista tiene complejidad temporal $O(c^2)$

Reconstrucción de las Rutas:

- Se tiene un máximo de n rutas. Por cada una:
 - Se parte de la posición correspondiente al espía y se itera hasta alcanzar el sumidero.
 - En cada iteración (c como máximo)
 - Se recorre la lista flujo para encontrar el primer elemento con igual origen y flujo positivo. $O(c)$
 - Se agrega la ciudad correspondiente a la ruta en construcción. $O(1)$
 - Se itera sobre la lista de centros. $O(n)$
 - Se compara la ciudad con el último origen antes de alcanzar el sumidero. De corresponder se agrega el nombre del centro a la ruta en construcción. $O(1)$

Por lo tanto, la reconstrucción cada ruta es $O(c^2)$ y, para todas las rutas, resulta $O(n \cdot c^2)$

Finalmente en la reconstrucción prevalece la complejidad temporal de la reconstrucción sobre el armado de la lista, por lo que todo el proceso resulta $O(n \cdot c^2)$. La misma complejidad que fue estimada en forma teórica.

¿Es posible expresar su solución como una reducción polinomial?

Se hizo una reducción del problema planteado al problema clásico de **flujo máximo en redes dirigidas con capacidades enteras**:

- Se modeló cada agente como una unidad de flujo desde un nodo fuente.
- Los centros de investigación fueron modelados como destinos inmediatamente previos hacia un nodo sumidero.
- Se modelaron las restricciones de tránsito por ciudad como división de nodos para limitar el paso.
- La bidireccionalidad de los vuelos se representó como dos aristas dirigidas independientes de sentido inverso.

La transformación propuesta se realiza en tiempo polinomial respecto al número de agentes, ciudades y centros.

Luego, dado que la **transformación del problema** original a una instancia de **flujo máximo** se hace en **tiempo polinomial**, y que el problema de flujo máximo se resuelve mediante el algoritmo de **Edmonds-Karp**, la solución propuesta es una reducción polinomial válida.

Parte 3: Las 2 jornadas de capacitación

Una nueva empresa se creó por la fusión de varias existentes. Por lo tanto se realizaron 2 jornadas de capacitación para todos sus equipos de desarrollo. Entre su personal existen “m” tipos de profesionales. Cada equipo de trabajo tiene como máximo 1 integrante con cada tipo de profesión entre sus miembros. Actualmente tienen “n” equipos de trabajo. En cada jornada pueden asistir no más de “p” personas por restricciones edilicias. Desean invitar a algunos tipos de profesiones para que vayan un día y el resto el otro. Una restricción adicional es que no puede pasar que todos los miembros de un equipo de trabajo asista todos juntos en un mismo día (para fomentar la integración).

Interpretación del problema

Sea $S = \{x_1, x_2, \dots, x_m\}$ un conjunto de m profesiones.

Sea $C = \{\{x_1, x_3\}_1, \{x_2, x_m\}_2, \dots, \{x_3, x_m\}_n\}$ un conjunto de n equipos, donde cada equipo tiene miembros con tipos de profesiones diferentes.

Queremos dividir S en dos jornadas (conjuntos) A y B tal que en cada jornada al menos 1 miembro de cada equipo en C . Además, estas jornadas A y B no pueden ser de tamaño mayor a p .

El problema es NP

Se puede construir un certificador en tiempo polinomial tal que certifique si una solución para una instancia es válida.

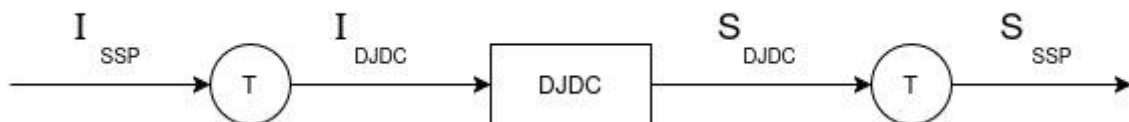
```
# S, C, p son instancias del problema.
# A, B es la solución a certificar.
certificador(S, C, p, A, B):
    ret false si  $|A| + |B| \neq |S|$   $O(1)$ 
    ret false si  $A \cap B \neq \emptyset$   $O(m)$ 
    ret false si  $A \cup B \neq S$   $O(m)$ 
    ret false si  $(|A| > p)$  ó  $(|B| > p)$   $O(1)$ 
    para cada equipo en C:  $O(n)$ 
        esta_en_A = esta_en_B = false  $O(1)$ 
        para cada profesion en equipo:  $O(m)$ 
            esta_en_A si  $\text{profesion} \subseteq A$   $O(1)$ 
            esta_en_B si  $\text{profesion} \subseteq B$   $O(1)$ 
            ret false si (no esta_en_A) ó (no esta_en_B)  $O(1)$ 
    ret true  $O(1)$ 
```

El problema es NP-Hard

Para demostrar que el problema de las 2 jornadas de capacitación es NP-H vamos a reducir el problema NP-H *Set Splitting Problem* a este. Es decir, queremos demostrar que:

$$SSP \leq_p DJDC$$

Esto lo logramos siguiendo el siguiente gráfico:



Si es posible obtener una solución válida para toda instancia de *SSP* a partir de esa doble transformación, demostramos que nuestro problema es igual o más difícil que uno NP-H, por lo que nuestro problema también lo será.

A continuación se presenta un pseudocódigo con transformaciones polinomiales para la resolución de *SSP* mediante el algoritmo *DJDC*.

```

transformacion( $S_{SSP}$ ,  $C_{SSP}$ ) :
     $S_{DJDC} = copia(S_{SSP})$   $O(m)$ 
     $C_{DJDC} = copia(C_{SSP})$   $O(n * m)$ 
     $p_{DJDC} = |S|$   $O(1)$ 
     $sol_{DJDC} = DJDC(S_{DJDC}, C_{DJDC}, p)$ 
     $sol_{SSP} = sol_{DJDC}$   $O(1)$ 
    ret  $sol_{SSP}$   $O(1)$ 
  
```

Esta transformación es válida, ya que el problema de las 2 jornadas de capacitación es una variante de *SSP* con una restricción agregada para el tamaño de *A* y *B*. Si logramos que esa restricción no altere la solución, como lo hecho en el pseudocódigo, vemos que ambos problemas son idénticos.

El problema es NP-C

Dado que demostramos que el problema pertenece a NP y pertenece a NP-H, podemos afirmar que también pertenece a NP-C.

Set splitting problem pertenece a NP-C

Para demostrar que el problema "Set splitting problem" pertenece a NP-C, debemos demostrar que pertenece a NP (existe un certificador que en tiempo polinomial puede verificar cualquier par de instancia y evidencia), y que pertenece a NP-H (podemos reducir polinomialmente un problema NP-H a este).

Set-Splitting Problem es NP

El problema “Set-splitting problem” consiste en lo siguiente: dado un set finito S de n elementos, y una familia F de m subsets de S , ¿existe una partición de S en dos tal que ninguno de los subsets de F están completamente en S_1 o S_2 ? Se puede construir un certificador en tiempo polinomial tal que certifique si una solución para una instancia es válida.

```
#S es un set finito, F es una familia de subsets de S
#S1 y S2 son la partición sugerida (certificado)
certificador (S, F, S1, S2)
    por cada subconjunto A en F:
        ret false si A ⊆ S1
        ret false si A ⊆ S2
    ret true
```

$O(m)$
 $O(n)$
 $O(n)$
 $O(1)$

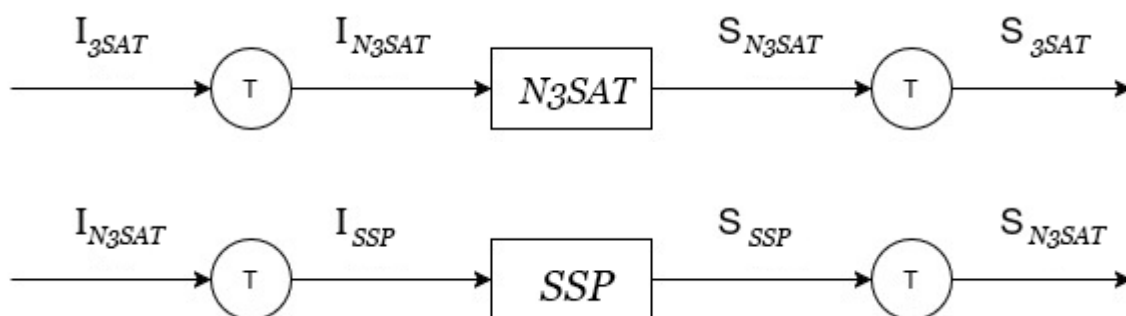
La complejidad temporal de este certificador es $O(n*m)$, así que se puede certificar en tiempo polinomial si una solución para una instancia es válida. Por lo tanto, el problema pertenece a NP.

Set-Splitting Problem es NP-Hard

Para demostrar que el problema “Set-splitting problem” es NP-H vamos a reducir el problema NP-H “3-SAT” a este. Es decir, queremos demostrar que:

$$3SAT \leq_p SSP$$

Para eso primero nos conviene reducir el problema 3-SAT a NAE-3-SAT. Es decir, vamos a aplicar las siguientes transformaciones:



Reducción de 3-SAT a NAE-3-SAT

El problema NAE-3-SAT es idéntico a 3-SAT, con el agregado de no admitir los mismos 3 valores por cláusula (es decir, en cada cláusula debe haber como mínimo un valor verdadero y uno falso).

A continuación se presenta un pseudocódigo con transformaciones polinomiales para la resolución de 3-SAT con n cláusulas mediante el algoritmo NAE-3-SAT

```

transformación( $I_{3SAT}$ )
     $I_{NAE3SAT} = copia(I_{3SAT})$   $O(n)$ 
    para cada cláusula  $(x \vee y \vee z)$  en  $I_{NAE3SAT}$ :  $O(n)$ 
         $s = variable\ auxiliar$   $O(1)$ 
        agregar variable en dos cláusulas de 4 literales  $O(1)$ 
        y reemplazar:
         $(x \vee y \vee z) \rightarrow (x \vee y \vee z \vee s) \wedge (\neg x \vee \neg y \vee \neg z \vee \neg s)$ 
    reducir cada cláusula a 3 literales:  $O(1)$ 
     $(a \vee b \vee c \vee d) \rightarrow (a \vee b \vee s) \wedge (c \vee d \vee \neg s)$ 
     $sol_{NAE3SAT} = NAE3SAT(I_{NAE3SAT})$   $O(1)$ 
     $sol_{3SAT} = sol_{NAE3SAT}$   $O(1)$ 
    ret  $sol_{3SAT}$ 

```

Demostramos que podemos resolver 3-SAT como una instancia de NAE-3-SAT mediante transformaciones polinomiales, por lo que NAE-3-SAT pertenece a NP-Hard.

Reducción de NAE-3-SAT a SPP

Ahora que demostramos que NAE-3-SAT pertenece a NP-H, podemos hacer la reducción polinomial a Set splitting problem para ver que este también pertenece a NP-H.

A continuación se presenta un pseudocódigo con transformaciones polinomiales para la resolución de NAE-3-SAT con n variables y m cláusulas mediante el algoritmo SSP

```

transformación( $I_{NAE3SAT}$ )
     $S = set\ con\ las\ n\ variables\ de\ I_{NAE3SAT}\ y\ sus\ negadas$   $O(n)$ 
     $F = familia\ de\ subsets\ de\ S$ 
    agregar todas las variables y sus negadas a  $F$  como un subconjunto  $\{x, \neg x\}$   $O(n)$ 
    agregar todas las cláusulas de  $I_{NAE3SAT}$  a  $F$  como subconjunto de 3 literales  $O(m)$ 
     $sol_{SSP} = SSP(S, F)$ 
     $sol_{NAE3SAT} = sol_{SSP}$   $O(1)$ 
    ret  $sol_{NAE3SAT}$   $O(1)$ 

```

Si existe una partición válida de S (S_1 y S_2) para esa instancia de set splitting problem, entonces eso significa que cada cláusula tiene al menos un literal en S_1 y otro en S_2 . Es decir, en cada cláusula de NAE-3-SAT, no todos los literales tienen el mismo valor de verdad. Esto es así porque, al incluir los sets de cada variable $\{x, \neg x\}$ en F , nos aseguramos que no puedan estar los dos en una misma partición de

S, y al agregar cada cláusula como un subconjunto de 3 literales a F, si todas las cláusulas están divididas (no hay ninguna cláusula enteramente en S1 o en S2), nos aseguramos que en cada cláusula haya al menos un literal falso y uno verdadero.

Entonces, si el Set splitting problem tiene solución, eso significa que cada cláusula de NAE-3-SAT tiene literales con distintos valores de verdad.

Demostramos que podemos resolver NAE-3-SAT como una instancia de SSP mediante transformaciones polinomiales, por lo que SSP pertenece a NP-Hard.

Como probamos que SSP pertenece a NP y también a NP-H, podemos decir que **SSP pertenece a NP-Complete**.

Qué pasa si se encuentra un método eficiente

Si una persona afirma tener un método eficiente para responder si es posible o no cualquiera sea la instancia del problema de las dos jornadas de capacitación, entonces esta persona afirma tener un método eficiente para resolver cualquier problema NP.

Esto es así dado que, por definición, cualquier problema NP se puede reducir en tiempo polinomial en un problema NP-C.

Que pueda resolverse cualquier problema NP en tiempo polinomial significa que $NP \subseteq P$. Como ya sabemos $P \subseteq NP$. Entonces, esto significa que se demostraría que $NP = P$.

Esto conlleva a que todos los problemas cuya decisión puede verificarse rápidamente también pueden resolverse eficientemente, trayendo muchos beneficios por un lado, pero también muchos problemas por el otro, como por ejemplo, problemas de seguridad informática.

Qué se puede decir de un problema que se reduce a este problema

Si un problema x se puede reducir polinomialmente al problema de “Las 2 jornadas de capacitación”, podemos asegurar que este problema x no es más difícil de resolver que nuestro problema. Que si resolvemos nuestro problema, también podemos resolver el problema x. Con otras palabras, podemos decir que nuestro problema es al menos tan difícil como x.

No podemos asegurar nada más, no sabemos si sí o no x pertenece a P, NP, NP-H o NP-C.

Referencias

1. Podberezski. V.. "Apunte TDA - Backtracking y branch and bound." Google Drive, 2025.
<https://docs.google.com/document/d/13SR2x7yf2zlo5tKztz6xQ6tkv12U4fxTlTrPzoiQBKU/edit?tab=t.0#heading=h.nrnw03t7conb>
2. Python heapq — Heap queue algorithm <https://docs.python.org/3/library/heapq.html>
3. Stirling Aproximation. https://en.wikipedia.org/wiki/Stirling%27s_approximation
4. Podberezski. V.. "Apunte TDA -. Redes de Flujo" Google Drive, 2025.
<https://docs.google.com/document/d/1bnQkeYNTriUmJTsj4GBwo4Qjuvkwh2ah2M3nB2mgSw0/edit?tab=t.0#heading=h.r0gqmbaik2gv>
5. Podberezski. V.. "Apunte TDA - Clases de complejidad" Google Drive, 2025.
https://docs.google.com/document/d/1Q2MpM2Zf-Frdg5kHkGec_NtgSaR1yTRoq_pvQ59km5w/edit?tab=t.0#heading=h.nrnw03t7conb
6. Introduction to Algorithms – Cormen, Leiserson, Rivest, Stein “Capítulo 24: Maximum Flow” MIT Press (2022) ISBN: 9780262046305
7. Wikipedia “Not-all-equal 3-satisfiability”
https://en-m-wikipedia-org.translate.goog/wiki/Not-all-equal_3-satisfiability?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc
8. Wikipedia "Set splitting problem"
https://en.wikipedia.org/wiki/Set_splitting_problem

1º Reentrega

Parte 1: La subasta de pizza

Pseudocódigo Backtracking

```
supera_restricciones(Seleccionados, Candidato, m, Restricciones):  
    Si el último de Seleccionados está entre las Restricciones del Candidato:  
        Retornar Falso  
    Si el Candidato está entre las Restricciones del último de Seleccionados:  
        Retornar Falso  
    Si el Candidato ocuparía el último lugar de los seleccionados: # |seleccionados| == m-1  
        Si el primero de los Seleccionados está entre las Restricciones del Candidato:  
            Retornar Falso  
        Si el Candidato está entre las Restricciones del primero de Seleccionados :  
            Retornar Falso  
    Retornar Verdadero  
  
bt_rec(seleccionados, m, n, restricciones, resultados):  
    si longitud(seleccionados) es igual a m:  
        agregar seleccionados a resultados y retornar  
  
    llamar a bt_rec(seleccionados + [None], m, n, restricciones, resultados)  
    #agregando un posible asiento vacío  
  
    para i desde seleccionados[0] + 1 hasta n:  
        si i ya se encuentra en seleccionados: #El evaluado ya se sentó  
            continuar (se poda)  
        si i no supera_restricciones(seleccionados, i, m, restricciones):  
            continuar (se poda)  
        llamar bt_rec con el siguiente de seleccionados  
  
backtracking(m, n, restricciones):  
    sea resultados una lista vacía  
    para i de 0 a n-1:  
        llamar bt_rec([i], m, n, restricciones, resultados)  
    retornar resultados  
  
buscar_max_en_rotaciones(factibles, ofertas):  
    sea máximo inicialmente un valor negativo  
    sea seleccionados una lista vacía  
  
    para cada estado factible en factibles:  
        repetir m veces:  
            ganancia_actual es 0  
            para cada porción desde la primera hasta m:  
                sea invitado el invitado sentado en la porción  
                si el invitado no es una silla vacía:  
                    ganancia_actual += oferta hecha del invitado por la porción  
            si ganancia_actual > máximo:  
                máximo = ganancia_actual  
                seleccionados = copia de factible  
            rotar factible a la izquierda #mover primer elemento al final  
    retornar máximo y seleccionados
```

```
sea m la cantidad de porciones
sea nombres los nombres de los invitados
sea ofertas las ofertas de los invitados
sea restricciones las enemistades de cada uno
factibles = backtracking(m, |nombres|, restricciones)
máximo, seleccionados = buscar_max_en_rotaciones(factibles, ofertas)
imprimir el máximo y los seleccionados
```

Pseudocódigo Branch & Bound

```
f_v_estimado(v_r_anterior, persona, silla, ofertas, sel, m, n):
    Crear un conjunto con los invitados ya asignados
    Sea v_e el valor acumulado v_r_anterior
    Crear una lista de máximas ofertas de tamaño |sillas restantes|

    a cada silla de máximas ofertas, se le da el mayor valor ofrecido por esa silla

    Sumar al valor v_e la suma de todos los valores en máximas ofertas

    Si la silla está vacía, retornar v_e
    Si no, retornar v_e más la oferta de persona para la silla actual

f_v_real(v_r_anterior, persona, silla, ofertas):
    retornar v_r_anterior sumado a la oferta de persona para la silla pasada

branchbound(m, n, restricciones, v_e_inicial):
    crear un heap de máximos que guarda tuplas (v_estimado, v_real, lista de personas)
    sea valor máximo actualmente 0
    sea la selección máxima una lista vacía

    por cada persona:
        calcular valor estimado v_e_p de sentarlo en la primera silla con f_v_estimado
        calcular el valor real v_r_p de sentarlo en esa silla con f_v_real
        si el valor estimado es menor al máximo hasta el momento, podar
        si el valor real es mejor al máximo actual:
            actualizar valor máximo y selección máxima
        insertar la tupla (v_e_p, v_r_p, [persona]) al heap
    mientras el heap no esté vacío
        extraer (v_e, v_r, lista) del heap
        si las sillas están llenas o v_e es menor al máximo, continuar
        incluir al heap el caso de silla vacía
        para cada invitado i que no haya sido asignado todavía:
            si i no cumple las restricciones con lista
                pasar a la siguiente iteración (podar)
            calcular valor estimado de sentar i en la silla actual con f_v_estimado
            si el valor estimado de sentar a i es menor que el máximo
                pasar a la siguiente iteración (podar)
            calcular el valor real de sentar a i en la silla actual con f_v_real
            agregar i a la lista de personas sentadas
            si el valor real con i supera al máximo:
                actualizar valor máximo y selección máxima
            insertar al heap el valor estimado, real y lista obtenida al sentar a i

    retornar valor y selección máximos
```

```
sea m la cantidad de porciones
sea nombres los nombres de los invitados
sea ofertas las ofertas de los invitados
sea restricciones las enemistades de cada uno
sea mejores_por_porcion una lista vacia de tamaño m
para cada elemento de mejores_por_porcion:
    aplicar la mejor oferta en esa posición
sea suma_mejores la suma de todos los elementos de mejores_por_porcion
máximo, seleccionados = branchbound(m, |nombres|, restricciones, suma_mejores)
imprimir el máximo y los seleccionados
```

Complejidad temporal Backtracking:

Para la solución por Backtracking se utiliza una función límite que en cada uno de los estados del problema o nodos, poda los que no cumplen con ciertas restricciones, estas restricciones son:

- Verificación de no haber sido seleccionado:
Los seleccionados se guardan en un diccionario, se verifica en $O(1)$.
- Verificación de no ser odiado por el anterior:
Esto es $O(1)$ ya que las restricciones están almacenadas en diccionarios.
- Verificación de no odiar al anterior:
De la misma forma por estar implementado con diccionarios verifica no odiar al anterior en $O(1)$.

Así la poda que realiza backtracking es $O(1) + O(1) + O(1) = O(1)$

Podemos concluir que la complejidad temporal de realizar la función backtracking es $O(n! n)$

(*) Para cada uno de los $n! n$ factibles se evalúa las m posibles rotaciones. Cada una de las m evaluaciones implica recorrer los m elementos para obtener la ganancia de esa rotación. Por lo tanto la complejidad temporal de evaluar todas las rotaciones es $n! n m^2$.

Entonces al utilizar backtracking la complejidad es realizar la exploración de las posibles combinaciones mas probar cada rotación para cada combinación.

Esto es $O(n! n) + O(n! n m^2) = O(n! n m^2)$

(*) Aclaración para reentrega

Parte 2: El traslado de información secreta

Variables de entrada

En la sección [Complejidad](#) se define el significado de las variables n , s y c .

Cota de *Cantidad de Agentes*

Del enunciado:

- “Contamos con un n agentes... Cada uno de ellos debe llevar la información a alguno de los n centros de investigación.”
- “Estos centros se encuentran cada uno en diferentes ciudades.”

Por lo tanto si:

- cantidad de agentes = cantidad de centros
- cada centro está en una ciudad
- no hay más de un centro en una misma ciudad

Entonces podemos asegurar que: **cantidad de ciudades $\geq$ cantidad de agentes**

Programa

Se corrige bug. Durante la última modularización, la llamada a la función `flujo_minimo()` quedó erróneamente con el parámetro `grafo_original` en lugar de `grafo_residual`.

Parte 3: Las dos jornadas de capacitación

La definición de C había quedado incorrecta al querer ser claro a la hora de definir el problema y sin darse cuenta mostrar una definición errónea de equipos de longitud 2. Se sugiere el cambio por una definición más abstracta:

Sea $C = \{E_1, E_2, \dots, E_n\}$ un conjunto de n equipos, donde cada E_i es un conjunto que cumple que $E_i \subseteq S$ y $E_i \neq \emptyset$. Es decir, cada equipo tiene miembros con tipos de profesiones diferentes.